SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I Y SAI GOPAL REDDY (192524083) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
sai gopal reddy
192524083

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

LEVEL 4 MODEL ANSWERS

The following responses address every task in the five scenarios. Each recommendation is tied to the scenario's constraints, trade-offs, and safety or performance requirements.

Question 1: Emergency Drone Routing

Greedy Best-First Search

The route map can be represented as waypoints connected by flyable paths. Greedy Best-First Search (GBFS) ranks each frontier waypoint only by h(n), here the straight-line distance to the medicine destination, and expands the waypoint that appears closest. This can find a plausible route quickly and may examine fewer locations when the estimate points toward the goal. It ignores the cost already paid, however. A short-looking path may cross flooded streets, rough terrain, or severe weather, so it can be slower and more dangerous than a longer route. It can also follow a misleading corridor or dead end and has no guarantee of finding the minimum-delay or minimum-risk route.

A* Search and the cost estimate

A* evaluates f(n) = g(n) + h(n). The path cost g(n) should accumulate the estimated flight time and a calibrated risk penalty for the route so far; blocked paths are excluded. The heuristic h(n) should estimate the remaining cost in the same units. For a time objective, straight-line distance divided by the drone's maximum feasible speed is a useful lower bound. On a fixed map with non-negative edge costs and an admissible, consistent heuristic, A* finds a least-cost route. It usually explores more states and keeps more search data in memory than GBFS, but it accounts for both progress and remaining cost.

Heuristic accuracy and uncertainty

A closer estimate can reduce unnecessary expansions, but it must match the objective. Straight-line distance is a lower bound on distance; by itself, it is not a time estimate when wind, terrain, or speed limits vary. A weak but admissible time heuristic preserves A*'s optimality while possibly expanding more nodes. If h(n) overestimates the remaining cost, A* may return a fast answer but can lose its optimality guarantee. GBFS depends entirely on h(n), so an inaccurate estimate can dominate every decision. The system should use current weather and terrain data for g(n), and retain a conservative lower-bound heuristic for A*.

Dynamic blocked paths

When a live update reports a blocked road or unsafe weather, the planner must mark the corresponding edge unavailable or increase its cost, invalidate any route that uses it, and plan again. GBFS is simple to rerun, but a fresh search discards previous work. A* can also restart, or an incremental method such as D* Lite can reuse information from the last plan and repair only the affected part of the route. Frequent updates improve safety but add computation and can cause repeated detours; the drone should validate each next segment against the latest data before committing to it.

Recommendation

I would use A* for the initial route and an incremental A*-based planner such as D* Lite when new blockages arrive. This balances accumulated delay and risk with an admissible remaining-time estimate, while reusing search information after changes. Safety rules should remove prohibited routes rather than treating every hazard as an ordinary small cost. Risk penalties, a computation deadline, and a verified safe fallback or return route make the plan useful under urgency. A* is optimal only for the current known map and chosen cost model; continuous sensing and replanning are still necessary because the real environment can change after a route is selected.

Question 2: Smart-City Traffic Signal Optimization

Optimization model and search size

Represent a candidate plan by the phase order, green-time split, cycle length, and coordination offset for each intersection over a planning horizon. Let W be total vehicle waiting time, F fuel consumption, and C a congestion measure such as queue length or spillback. The objective can be written as minimize J = alpha*W + beta*F + gamma*C, with normalized terms and weights set by the city. Hard constraints enforce safe phase transitions, minimum and maximum green times, pedestrian crossing time, intersection capacity, and legal signal sequences. The search space grows roughly as the product of the choices at hundreds of junctions. Intersections are coupled, traffic changes while a plan is evaluated, and each full evaluation may require a simulation. Exhaustive or uninformed search is therefore too slow and its result may be stale before it is deployed.

Hill Climbing

Hill Climbing starts with a valid timing plan, generates nearby plans by changing one or a small group of signal settings, evaluates J, and adopts a neighbor with a lower cost. It is easy to implement and can improve a current schedule quickly. It does not systematically explore the whole space, so it can stop at a locally good plan even when a much better coordinated plan exists. A greedy update at one junction can also shift queues into a neighboring junction. The city should evaluate candidate changes over representative traffic windows rather than trusting a single noisy measurement.

Local maxima, plateaus, and ridges

Because this task minimizes cost, the common failure is a local minimum; in a maximization formulation it is the corresponding local maximum. For example, one light's green time may be locally optimal even though changing several connected lights together would reduce network delay. A plateau occurs when several timing choices produce nearly the same measured cost, such as when rounded traffic counts hide small improvements; the search has no clear direction. A ridge occurs when improvement requires coordinated changes across multiple junctions, while changing only one signal temporarily worsens the score. These effects are realistic because queues move through a connected network rather than ending at one light.

Simulated Annealing and Genetic Algorithms

Simulated Annealing (SA) explores one candidate plan at a time. It accepts an improving change and may also accept a worse change with probability e^(-delta/T), where delta is the increase in cost and T is a temperature that gradually cools. Early exploration can cross a local barrier; later cooling focuses the search. A Genetic Algorithm (GA) maintains a population of plans and uses selection, crossover, and mutation to explore different combinations in parallel. Both can escape some local traps, but SA requires a cooling schedule and GA needs population and mutation settings; either can consume substantial time if each fitness evaluation simulates the full city.

Recommendation

For continuous operation, I recommend a hierarchical, distributed SA over regional groups of intersections, with boundary coordination between regions. Start each rolling-horizon search from the current safe plan, use a bounded number of iterations, and keep all hard safety constraints enforced. This reduces the search handled at once, lets the controller respond to new traffic counts, and permits temporary worse moves that can escape a local minimum. A GA can be useful offline to discover strong initial timing templates, but the bounded SA is easier to warm-start and adapt at runtime. The system should monitor performance and revert to a verified legal plan if the time budget expires or sensor data becomes unreliable.

Question 3: Mars Rover Online Search

Why online search is required

An offline planner assumes it knows the relevant map and can compute a complete route before movement begins. The rover starts with an incomplete map, has limited sensing, and may discover a crater, loose ground, or blocked passage only when it approaches. A route planned from assumptions can therefore be unsafe or impossible. An online search agent repeatedly observes its surroundings, updates its knowledge, chooses a reachable action, and plans again. It can make useful progress without waiting for a complete map, which is important when communication with Earth is delayed.

Partial observability and changing terrain

The rover's sensors see only a limited area and may be noisy; its location and the traversability of distant cells are uncertain. A rock may hide a drop, and slope or wheel-slip estimates may change as new measurements arrive. Hazards, lighting, and terrain interactions make edge costs uncertain. The rover also has strict energy, time, and communication limits, so an apparently short path may be infeasible if it risks immobilization or consumes the return reserve. The planner should distinguish known safe, known unsafe, and unknown terrain rather than treating every unobserved cell as open ground.

Building and updating the internal model

The rover can maintain a local occupancy or topological map with a confidence value for each cell or edge, along with its estimated pose, terrain type, hazard locations, and travel cost. After each camera, range, or wheel-sensor observation, it updates the affected map cells and revises their traversability and risk estimates. It records sample goals and unexplored frontiers, then replans from its current pose. New evidence can clear a suspected hazard or mark a previously open route as blocked. Unknown areas should receive conservative costs until the rover has sensed enough to classify them.

Online, uninformed, and informed search

Breadth-First Search finds a fewest-step path only when the map is known and actions have equal cost; it ignores terrain risk. Depth-First Search uses little memory but can wander far from a goal and does not ensure a low-cost route. A* uses a heuristic to reduce exploration on a known map and can be effective when its costs and heuristic are sound, but a static plan becomes stale as discoveries arrive. Online search combines planning with action and observation. Methods such as LRTA* learn estimates while moving, and incremental planners such as D* Lite repair paths when the rover's map changes; they may detour or replan repeatedly, but they adapt to what the rover actually observes.

Recommendation

Use a risk-aware online agent with a local map and an incremental heuristic planner such as D* Lite. The map should represent uncertainty, and unknown or hazardous terrain should be penalized or excluded until sensed. Before each move, a separate safety check should verify slope, clearance, energy reserve, and a feasible fallback route. Replanning after each meaningful observation lets the rover adapt without recomputing everything from scratch. This approach is more suitable than a one-time offline path because it responds to partial knowledge, while conservative costs and return-energy limits protect sample collection and rover survival. No search algorithm can guarantee safety if its sensors or model are wrong, so uncertainty must remain explicit.

Question 4: University Exam Timetabling as a CSP

CSP formulation

Let each course exam be an event c and define a variable X_c = (time slot, room). Its domain contains only feasible slot-room pairs: rooms with sufficient capacity and required accessibility, slots when the room and required faculty are available, and durations that accommodate the exam. If invigilators are assigned by the system, add a variable P_c whose domain is the available qualified faculty. The constraints are the rules that remove incompatible assignments. Two exams sharing a student must use different times; a room cannot host two exams in the same time block; room capacity must meet enrollment; and required faculty cannot be double-booked or assigned when unavailable. A prerequisite ordering becomes slot(A) < slot(B). Accommodation needs, including accessible rooms and extra time, must be represented in the domain and overlap constraints.

Hard and soft constraints

Student conflicts, room collisions, faculty availability, precedence, capacity, and required accommodations are hard constraints: a valid timetable cannot violate them. Preferences such as avoiding several exams on one day, using a preferred room, or balancing exam load are soft constraints. The system should first find an assignment satisfying every hard constraint, then minimize penalties for soft constraints. Keeping the two classes separate prevents a high preference score from ever outweighing student safety, access, or a required no-overlap rule.

Backtracking search

Backtracking assigns one exam variable at a time. It chooses a feasible slot-room pair, checks it against the partial timetable, and recursively assigns the next exam. If a later exam has no valid option, it removes the last assignment and tries another value. The Minimum Remaining Values heuristic selects the exam with the fewest legal choices first; degree centrality breaks ties in favor of exams that constrain the most others. Least Constraining Value tries the option that removes the fewest choices from other exams. These heuristics expose difficult conflicts early and make each assignment more useful than arbitrary ordering.

Constraint propagation

After assigning an exam, forward checking removes incompatible time-room pairs from the domains of unassigned exams that share students, rooms, or faculty. If any domain becomes empty, the current branch is impossible and the solver backtracks immediately. Arc consistency can repeat this pruning between related variables, while global all-different or capacity constraints can capture groups of exams that compete for the same room or period. Propagation does not eliminate the worst-case exponential complexity, but it prevents the solver from exploring many partial schedules that cannot be completed.

Scalability strategy and real-world issues

Large student-course conflict graphs, popular courses, limited halls, late enrollment changes, faculty absences, and unusual accommodations can create tightly connected constraints. Data quality and fairness also matter: an apparently valid schedule may still be unacceptable if it gives some students repeated early or late exams. I recommend a CSP solver using MRV, degree and least-constraining-value ordering, forward checking, and arc consistency, followed by soft-constraint optimization. Decompose loosely connected course groups where possible, but validate the full timetable globally. When room or enrollment data changes, repair only the affected region and rerun global checks for student conflicts and capacity.

Question 5: Minimax and Alpha-Beta for a Strategy Game

Minimax model

For a deterministic, turn-based, two-player zero-sum abstraction, each game state is a node and each legal action is an edge. The AI is the MAX player, which selects the action with the highest backed-up value; the opponent is MIN, which is assumed to choose the reply that lowers the AI's value. Terminal wins, losses, and draws receive large positive, negative, and neutral utility values. For a real-time strategy game, the tree can use meaningful macro-actions, such as defend, expand, or attack, at regular decision ticks. Standard Minimax is only an approximation when actions are simultaneous, outcomes are random, or information is hidden; those cases may require chance nodes or opponent-belief models.

Computational challenges

If each state has b possible actions and the search looks ahead d decisions, the tree may contain on the order of b^d nodes. Real-time strategy games have many units, commands, resource choices, and timing options, so b can be very large. State generation and evaluation also consume time, and a depth limit can miss a delayed consequence or create a horizon effect. A depth-first implementation uses roughly O(b*d) memory, but its computation still grows exponentially. The strict decision deadline makes an unbounded exhaustive search impractical.

Alpha-Beta pruning

Alpha-Beta keeps alpha, the best value MAX can already guarantee, and beta, the best value MIN can already guarantee. At a MAX node it raises alpha; at a MIN node it lowers beta. Once alpha >= beta, the remaining children of that node cannot change the choice already available to an ancestor, so the solver prunes them. For example, if MAX already has a move worth 5 and another branch is controlled by MIN, once MIN finds a reply worth 3, MAX would never select that branch, regardless of its unexplored replies. Alpha-Beta returns the same decision as Minimax at the same depth, but good move ordering can reduce work toward O(b^(d/2)); poor ordering leaves the O(b^d) worst case.

Evaluation function

At a depth limit, use a weighted score such as V(s) = w1*unit_advantage + w2*resource_advantage + w3*objective_control + w4*territory_and_mobility - w5*immediate_threat. Unit advantage should account for health and strategic value, while resources and technology estimate future options. Objective control reflects progress toward winning conditions; mobility and territory measure access and positioning; threat captures imminent losses or danger. Terminal wins and losses should dominate ordinary scores. Normalize component ranges, tune weights with repeated matches, and avoid counting the same benefit twice so a large resource value does not hide an immediate threat.

Real-time recommendation

Use iterative-deepening Alpha-Beta with strong move ordering, a transposition table, and the evaluation function above. Iterative deepening searches depth 1, then deeper levels, and keeps the best move from the last fully completed iteration; if the time budget expires, the AI still has a legal action. Search macro-actions and replan at defined decision ticks to control branching. Retain hard safety checks for illegal or suicidal moves. This provides exact Minimax choice for the searched horizon while pruning many irrelevant branches. For simultaneous, stochastic, or hidden-information parts of an RTS game, combine this adversarial search with chance or belief-state reasoning rather than assuming standard Minimax fully captures the game.